

ЗАДАЧА РОЗИ – НАСЛЕДЯВАНЕ

Задачата демонстрира модификатор за достъп `protected` в родителския клас и наследника, наследяване на конструктор с `base`, презаписване на метод с `virtual -> override`.

```
Microsoft Visual Studio Debug Console

Цъфтящите "Розоцветни" са изключително красиви и имат благоуханен аромат.          РОДИТЕЛ
Oubaitori е японски идиом, според който цветята, както и хората, имат свое собствено темпо на цъфтеж и развитие.
~~~~~
Розите цъфтят през цялото лято с деликатен аромат.                                НАСЛЕДНИК 1
Сорт "Императрица Фара" е с малко бодли.
Oubaitori е японски идиом, според който цветята, както и хората, имат свое собствено темпо на цъфтеж и развитие.
~~~~~
Розите цъфтят през цялото лято с деликатен аромат.                                НАСЛЕДНИК 2
Сорт "Zephirine Drouhin" е с напълно гладко стебло без бодли.
Oubaitori е японски идиом, според който цветята, както и хората, имат свое собствено темпо на цъфтеж и развитие.
~~~~~
Розите цъфтят през цялото лято с деликатен аромат.                                НАСЛЕДНИК 3
Сорт "Rosa rugosa" е с изключително много бодли.
Oubaitori е японски идиом, според който цветята, както и хората, имат свое собствено темпо на цъфтеж и развитие.
~~~~~

D:\Exercises\11 class\Inheritance\classwork_example\bin\Debug\net8.0\Flowers.exe (process 70204) exited with code
Press any key to close this window . . .
```

Първо създаваме родителския клас:

```
class Flower
{
    3 references
    public string Name { get; set; }

    1 reference
    protected string Fragrance { get; set; } = "благоуханен"; невидим в Main(), но достъпен в наследниците

    2 references
    public Flower(string name)
    {
        Name = name;
    }

    4 references
    public void Oubaitori() // общ за родител и наследници
    {
        Console.WriteLine("Oubaitori е японски идиом, според който цветята, както и хората, имат свое собствено темпо на цъфтеж и развитие.");
        Console.WriteLine(new string('-', 77));
    }

    5 references
    public virtual void Blooming() // ще бъде променен = презаписван от наследниците
    {
        Console.WriteLine($"Цъфтящите \"{Name}\" са изключително красиви и имат {fragrance} аромат.");
    }
}
```

Създаваме клас, който наследява:

```

class Rose : Flower наследява родителски клас
{
    is a
    2 references
    public string Thorns { get; set; }
    3 references
    public Rose(string name, string thorns) : base ( name ) наследява конструкторски параметър
    {
        Thorns = thorns; // имената вече са свързани в конструктора на родителя
    }
    3 references
    public void ThornsQuantity() // специфичен за наследника метод
    {
        Console.WriteLine($"Сопм \"{Name}\" е с {Thorns} богли.");
    }
    5 references презаписва
    public override void Blooming() // променен върху родителския
    {
        fragrance = "геликамен";
        Console.WriteLine($"Розите цъфтят през цялото лято с {fragrance} protected аромат.");
    }
}

```

Създаваме нови обекти в Main() – в случая един родител и 3 наследника:

```

static void Main(string[] args)
{
    Flower f1 = new Flower("Розоцветни"); // РОДИТЕЛ
    f1.Blooming();
    f1.Oubaitori(); //общ за всички

    НАСЛЕДНИЦИ
    Rose r1 = new Rose("Императрица Фара", "малко");
    r1.Blooming(); // презаписан върху родителския
    r1.ThornsQuantity(); // специфичен за наследника
    r1.Oubaitori();

    Rose r2 = new Rose("Zephirine Drouhin", "напълно гладко стебло без");
    r2.Blooming();
    r2.ThornsQuantity();
    r2.Oubaitori();

    Rose r3 = new Rose("Rosa rugosa", "изключително много");
    r3.Blooming();
    r3.ThornsQuantity();
    r3.Oubaitori();
}

```

Целият код:

```

internal class Program
{
    static void Main(string[] args)
    {
        Flower f1 = new Flower("Розоцветни"); // РОДИТЕЛ
        f1.Blooming();
        f1.Oubaitori(); //общ за всички
    }
}

```

```

        Rose r1 = new Rose("Императрица Фара", "малко");
        r1.Blooming();           // презаписан върху родителския
        r1.ThornsQuantity();    // специфичен за наследника
        r1.Oubaitori();

        Rose r2 = new Rose("Zephirine Drouhin", "напълно гладко стебло без");
        r2.Blooming();
        r2.ThornsQuantity();
        r2.Oubaitori();

        Rose r3 = new Rose("Rosa rugosa", "изключително много");
        r3.Blooming();
        r3.ThornsQuantity();
        r3.Oubaitori();
    }
}

class Flower
{
    public string Name { get; set; }
    protected string fragrance { get; set; } = "благоуханен";

    public Flower(string name)
    {
        Name = name;
    }

    public void Oubaitori() // общ за родителя и наследниците
    {
        Console.WriteLine(@"Oubaitori е японски идиом, според който цветята, както и хората,
имат свое собствено темпо на цъфтеж и развитие.");
        Console.WriteLine(new string('~', 77));
    }

    public virtual void Blooming() // ще бъде променян = презаписван от наследниците
    {
        Console.WriteLine($"Цъфтящите \"{Name}\" са изключително красиви и имат
{fragrance} аромат.");
    }
}

class Rose : Flower
{
    public string Thorns { get; set; }
    public Rose(string name, string thorns) : base ( name )
    {
        Thorns = thorns; // имената вече са свързани в конструктора на родителя
    }

    public void ThornsQuantity() // специфичен за наследника метод
    {
        Console.WriteLine($"Copt \"{Name}\" е с {Thorns} бодли.");
    }

    public override void Blooming() // променен върху родителския
    {
        fragrance = "деликатен";
        Console.WriteLine($"Розите цъфтят през цялото лято с {fragrance} аромат.");
    }
}

```